

Multicore Programming

OpenMP - MPI - CUDA

Julio Zenelaj

April 18, 2025

1. Introduction

The scope of this project was the modification of a given sequential C script in order to implement the parallel programming models studied during the Multicore Programming course.

1.1. The Task

The task performed by the given script was pattern matching for DNA Sequences: given a sequence of characters representing a DNA sequence and a list of patterns, the program finds the index of the first occurrence of each pattern in the sequence using a brute-force approach. Furthermore, it calculates the total number of patterns found, and for each position of the main DNA sequence, the number of patterns that are aligned covering that position. In the end, a series of checksums are computed to ensure the correctness of the parallel program with respect to the sequential one.

The DNA sequence and list of patterns are generated at runtime, based on a set of parameters that define them:

- *seq_length*: Length of the main DNA sequence.
- *prob_G*: Probability of occurrence of G-nucleotides.
- *prob_C*: Probability of occurrence of C-nucleotides.
- *prob_A*: Probability of occurrence of A-nucleotides.
- *pat_rng_num*: Number of random patterns.
- *pat_rng_length_mean*: Mean length of the patterns.
- *pat_rng_length_dev*: Length deviation of those patterns.
- *pat_samples_num*: Number of patterns that are samples of the original sequence.
- *pat_samp_length_mean*: Mean length of the sample patterns.
- *pat_samp_length_dev*: Length deviation of the sample patterns.
- *pat_samp_loc_mean*: Mean location of the beginning of the samples.
- *pat_samp_loc_dev*: Deviation from the start location.

- *pat_samp_mix*: B[efore] | A[fter] | M[ixed]
- *long_seed*: A random seed for the generation of different and reproducible cases for the same arguments.
- *thread_count* (Added)[OpenMP, OMP+MPI]: The number of threads to be used by the OpenMP model.
- *grid_size* (Added)[CUDA]: The number of blocks in the CUDA grid.
- *block_size* (Added)[CUDA]: The number of threads per block in the CUDA grid.

1.2. Project Goal

The goal of this project is the modification of the given script, without changing the algorithm, utilizing the three parallel programming models studied during the course: the CPU parallelization model(s) (OpenMP/PThreads), the GPU parallelization model (CUDA) and the distributed computation model (MPI). The chosen implementations are:

- OpenMP
- OpenMP + MPI
- CUDA

	Input Size (in thousands)			
	250	500	1,000	2,000
Runtime (s)	172	349	702	1458

Table 1: Sequential Program Runtime Reference

2. OpenMP

2.1. Implementation

The implementation of the OpenMP model required a relatively small amount of changes, thanks to the high level interface that it offers for parallel programming. These are the changes that were performed:

- Parallelization of the initialization of structures within the allowed region.
- Parallelization of the main loop, with the given *thread_count* number of threads. Here, we perform buffer reduction on *seq_matches* to avoid

race conditions when writing to it. Reduction is possible since *seq_matches* is simply a buffer of counters. We also set *pat_matches* for addition reduction since it is a general counter. We also set the scheduling type as discussed below.

- Inside the loop, we mark the section in which writing is performed as atomic.
- This section corresponds to the usage of the *increment_matches(...)* function which was unrolled to avoid potential problems with parameter passing and to not utilize unnecessary memory since this is the only place this function gets used in.
- In the end, the checksum calculations are parallelized as reductions.

Changes that were considered but were not possible:

- Parallelizing the generation of the sequence. This was not possible as the sequence has to match the one of the sequential solution on a given seed and each time a random operation is performed the state of the random generator is changed; therefore, performing the generation in parallel would cause different random generator states for a given index of the sequence and the sequence would be different with respect to the one generated in the sequential case. Another workaround would be to use the index of the letter, combined with the global *long_seed*, but this change would need to be made also in the sequential script, which is not possible.
- Another change that was considered is the unrolling of the loops of the matching algorithm, but this is not possible since the second level loop has a number of steps that depends on the input, so it is not fixed and unrolling would not be possible.
- It was also considered using dynamic scheduling, but the results of tests showed that this choice would not be more performant than static scheduling in general. (as can be seen in Figure 1).

2.2 Results

Following are the results of different performance tests performed on the above solution.

Figure 1 shows the variance of runtime of different scheduling types. The lines in color correspond to the dynamic scheduling type, each representing a different batch size. The black line shows the static scheduling type. From this data, it appears that for smaller input sizes, the different scheduling choices have roughly similar results. However, on larger inputs, the differences are much more obvious, with

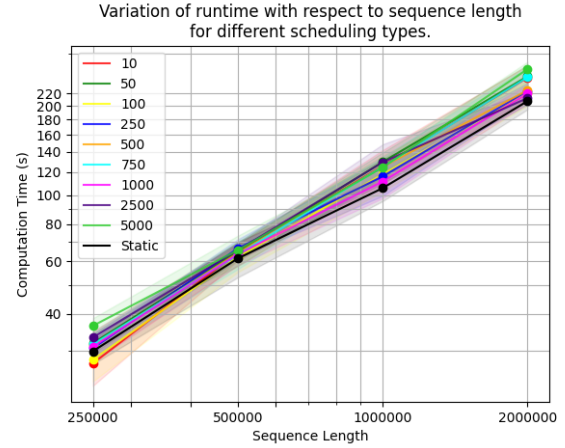


Figure 1: The runtime of the OpenMP implementation under different scheduling types.

static scheduling taking about 200 seconds on the largest input size and the slowest dynamic batch size choice taking about 240 seconds. It is then obvious why the choice of scheduling type matters and since static has better performance overall as well as better scalability, it was picked as the default scheduling type moving forward.

	Input Size (in thousands)			
Threads	250	500	1,000	2,000
1	215	435.6	702	1667.8
2	118.8	234.3	401.6	909.1
4	53.6	106.3	214.2	432.5
8	26.5	53.4	107.5	216.8

Table 2: Time Matrix OpenMP

	Input Size (in thousands)			
Threads	250	500	1,000	2,000
1	0.8	0.8	1	0.87
2	1.44	1.49	1.7	1.6
4	3.2	3.28	3.2	3.37
8	6.5	6.54	6.5	6.73

Table 3: Speedup Matrix OpenMP with respect to the sequential program runtime (Table 1)

	Input Size (in thousands)			
Threads	250	500	1,000	2,000
1	~	~	~	~
2	0.72	0.74	0.87	0.8
4	0.8	0.82	0.82	0.84
8	0.81	0.82	0.82	0.84

Table 4: Efficiency Matrix OpenMP

Lastly, we consider the scaling of our solution, by performing a matrix evaluation of the program and

considering the speedup and efficiency results.

The major factor that affected the speedup and efficiency results of the program is the use of a hard-enough task. On easier tasks the program performs much worse in terms of these metrics as the overhead of the sequential parts outweighs the benefits of parallelization.

The results of the tables show the presence of weak scaling of the program. The fact that efficiency increases as the number of threads increases can be considered as strong scaling, despite the fact that there is some margin from an optimal utilization of the threads.

3. OpenMP + MPI

3.1. Implementation

The addition of a distributed computing model demanded that the initial generation of the values be performed by the master node, to be then distributed to the worker nodes, which perform their task independently and finally the results get fed back to the master node, which calculates and outputs the final results. The master node is also a worker node. The distribution of work in this case consists in the splitting of the list of patterns between the nodes. All the nodes also receive the data needed to generate the sequence locally and they need to perform the check for their own share of patterns and then report the results.

The structure of the program is as follows:

- First of all, the no-change sections of the default sequential script demanded overwriting as the setup phase needed to be modified in such a way that the master node performs the initial generation of the main values.
- Once generation is done, the master node distributes the values to the other nodes through **MPI_Bcast**.
- Then, the patterns are distributed using **MPI_Scatterv** in order to split the patterns in an uneven fashion. The datatype used is a contiguous type of 10,000 characters. This was necessary because MPI does not perform transfers which have a send count higher than the maximum integer value, which was often crossed before using the new datatype. This is also why nodes need to receive a variable number of data; it depends on the splitting with this new datatype. Also, padding is often needed to match the size of this datatype.
- The data for the generation of the sequence is broadcast and each node generates the sequence locally.

- The main loop is changed with respect to the simple OpenMP implementation. Instead of calculating the main results in-place, since we have the data distributed among many nodes, we only calculate the local results for each node.
- After the main loop, all the results get sent to the master node (using **MPI_Gatherv**), which then calculates in parallel using OpenMP the final results and shows them.

3.2. Results

Below are some results of the tests performed on this solution.

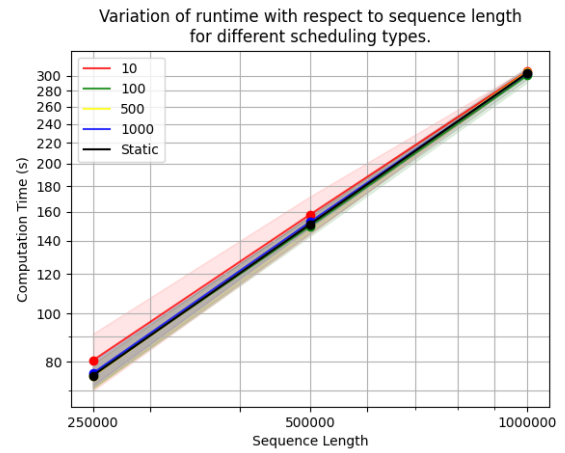


Figure 2: The runtime of the OpenMP+MPI implementation under different scheduling types.

Tests were run to verify the scheduling method of OpenMP on this implementation as well and as it shows in Figure 2, static scheduling is, not surprisingly, still the best choice.

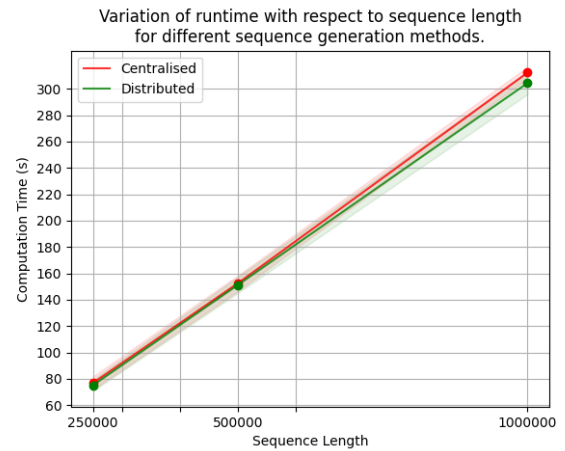


Figure 3: The comparison of runtime of the OpenMP+MPI when the sequence is generated by the master node (centralized) as opposed to when it is generated locally by each node (distributed).

Figure 3 shows the runtime differences when the

main sequence is generated by the main node as compared to when the seed and mix information is distributed and the sequence is generated independently by each node. The expectation is that, since the sequence can be of considerable size, then its distribution might be a serious bottleneck for the program. In addition, since the main node generates the sequence while the other nodes are idle, then there would be no additional delay if each node generated the sequence locally

Despite these expectations, we can see that the difference is not very considerable. However, with the increase in size of the sequence, it starts to become noticeable, so for scaling purposes it makes sense to perform the generation on each node separately.

Nodes	Input Size (in thousands)			
	250	500	750	1,000
1	174.8	350.7	533.1	711.3
2	117.1	229.8	343.2	564.4
3	83.7	163.9	241.7	325.6
4	67.6	130.5	193.8	260.1

Table 5: Time Matrix OpenMP+MPI

Nodes	Input Size (in thousands)			
	250	500	750	1,000
1	1	1	1	1
2	1.49	1.53	1.55	1.26
3	2.08	2.14	2.21	2.18
4	2.59	2.69	2.75	2.73

Table 6: Speedup Matrix OpenMP+MPI

Nodes	Input Size (in thousands)			
	250	500	750	1,000
1	1	1	1	1
2	0.75	0.76	0.78	0.63
3	0.7	0.71	0.74	0.73
4	0.65	0.67	0.69	0.68

Table 7: Efficiency Matrix OpenMP+MPI

The timing results for the MPI implementation show the lack of strong scaling as with the increase of the number of nodes, the speedup of the program does not keep up and efficiency drops. However, when it comes to the increase of the size of the problem, we see that the program gets more efficient the bigger the problem size, which might suggest the presence of weak scaling. But then this increase stops at the sequence size of 1 million characters. Considering the trend for other sizes, this drop in performance might be due to external limitations such as memory, caching, data transfer, a fluctuation in results etc. Therefore, a wider range of experiments is needed

to determine the presence of weak scaling in the program.

4. CUDA

4.1. Implementation

The implementation of GPU parallelization required the creation, maintenance and clearance of GPU-bound data structures and an exchange of data with the CPU-bound structures. The general organization of work of this problem is as follows:

- The CPU creates the necessary data structures and generates the data. Then, the GPU data structures are created and data is copied to the GPU.
- The GPU has a number of blocks, given as a parameter. Each block is assigned a (tentatively) equal number of patterns. The number of threads per block is also given as a parameter.
- Inside a block, each thread checks a (tentatively) equal number of sequence indices for the presence of patterns. This amount of indices is, however, not sequential but each thread checks a multiple of the *block_size*, offset by the respective thread index. This is done to exploit caching on the sequence.
- A global data structure is used to maintain the search results of the threads. This is because the sequential algorithm searches for the *first* appearance of a pattern in the sequence and this must be maintained also in the GPU-parallel version. Therefore, whenever a thread finds a matching, it checks if the index of that matching is lower than the one currently in the global buffer (if any), and if so, it overwrites it.
- Finally, this global buffer is copied to the CPU and the final results are calculated and output.

Changes that were tried but were not successful:

- Utilization of shared variables would have benefited the program as it would work as a cache, allowing the threads to access the data structures which are frequently used much faster. However, due to the fact that these major data structures are buffers of variable size, this was not possible because the amount of memory that shared structures need must be known at compile time.
- A pre-processing step was considered to be used in the kernel. Initially, the sum of the numerical value of the characters of each pattern was calculated on the CPU; let's call it a checksum. Each pattern has its checksum. On the GPU kernel, threads calculated the sum of characters in subsequences of length the length of the pattern

currently in consideration. If this sum matches the pattern's checksum, then the index of the subsequence is stored. Then the search is conducted normally on the indices that come out of the filtering step. The rationale of this idea is that computing sums in this way is a high throughput process with a low number of breaks, which would keep the threads utilization higher than during the search phase. This would hopefully offset the time lost during searching as well as lower the number of searches needed. It was also based on the idea that the probability of false positives is probably not that high for patterns with such a limited set of characters. One downside of this approach is that, due to the lack of dynamic buffers on the GPU, we need to keep a binary table of $seq_length \times pattern_count$ dimensions to sign the matches. In the end, this method did not give any speedup over the simple search therefore it was abandoned.

4.2. Results

These are the results of the CUDA program tests.



Figure 4: The change in runtime with respect to the number of blocks used (fixed number of threads).

As expected, the graph in Figure 4 initially shows a trend in decline as having more blocks than the number of SMs (in this case, 72), allows higher utilization of the cores as blocks can be swapped and then the trend continues in increase because having too many blocks forces some of them to wait idling. The most striking feature of this graph however, is the high variance at the minimum of 216 blocks. One explanation could be that these extrema are simply fluctuations of the machine. For the rest of tests, 288 was chosen as the default grid size since it is close to the minimum but with a much lower variance.

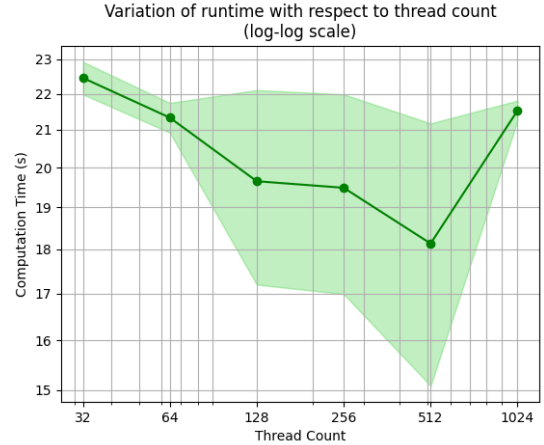


Figure 5: The change in runtime with respect to the number of threads used (fixed number of blocks).

Figure 5 shows as expected a decreasing trend in runtime with respect to the increase of the number of threads, except for the case of 1024 threads. We once again observe the problem of very high variance as the number of threads increases. The decreasing trend shows that the workload has properly been split among the threads but the high variance as the number of threads increases indicates that the organization of the threads is not optimal; there might be a lot of divergence in the work of threads, data transfer might be suboptimal, cache utilization might be suboptimal etc. The sudden increase in time when utilizing 1024 threads might indicate that at such a high number of threads per block, the problems mentioned above outweigh the benefits of splitting the work.

Threads	Input Size (in thousands)			
	250	500	1,000	2,000
64	11.5	20.6	35.2	69.1
128	10.9	18.4	35.3	68.4
256	9.2	20.1	35.1	67.6
512	10.6	20.1	36.6	70.2

Table 8: Time Matrix CUDA

The time matrix shows that for any number of threads, the runtime is roughly similar, showing that the way in which work has been split is not enough to make up for the delays in other parts of the program. It remains however a considerable improvement over the sequential program (Table 1) with speedups from 17 to 20 times in comparison.

5. Conclusion

We present three different implementations of parallelized versions of the given program, utilizing the three models of parallel computation studied in class: the CPU parallelization model, the GPU parallelization model and the distributed computation model. Each approach has its benefits and drawbacks: CPU parallelization (OpenMP) is a good choice for local parallelization, inherently on problems of smaller size; distributed computing (MPI) is useful for distributing large amounts of data among several nodes for local processing but the data transfer latency should be justified by the speedup of the parallel computing; GPU parallelization (CUDA) is good for high throughput tasks, such as calculations, but work should be well-organized and branches and jumps should be avoided in the program.